Code Review

1. Change History

Change Date	Modified Sections	Rationale
2026-10-18	2, 3	CP1: neither code review has run yet; both are scheduled for later checkpoints.
2026-10-26	2	CP2: ran the course's ESLint and detekt configurations locally (2 ESLint findings, justified in 2.4; none from detekt).
2026-10-27	2	CP2: recorded Codacy's run. It found 103 issues on `8e9f3f7`; commit `ceec5d9` fixed 8, and the other 95 are justified in 2.4.
2026-11-02	All	Shortened the wording after CP1 feedback.

2. Automated Code Review Results

Status at CP2: Codacy analyses `abhiverma13/TimeTravel` with its default coding standard and reported 103 issues on `8e9f3f7`. Commit `ceec5d9` fixed 8: 4 vulnerable dependencies (`qs`, `body-parser`, `proxy-addr`, in `backend/package-lock.json`), 2 `$null` comparisons in `scripts/run-backend.ps1`, and 2 hard-coded JWT secrets in `backend/tests/no-mock/auth.test.ts`. The other 95 are justified in 2.4.

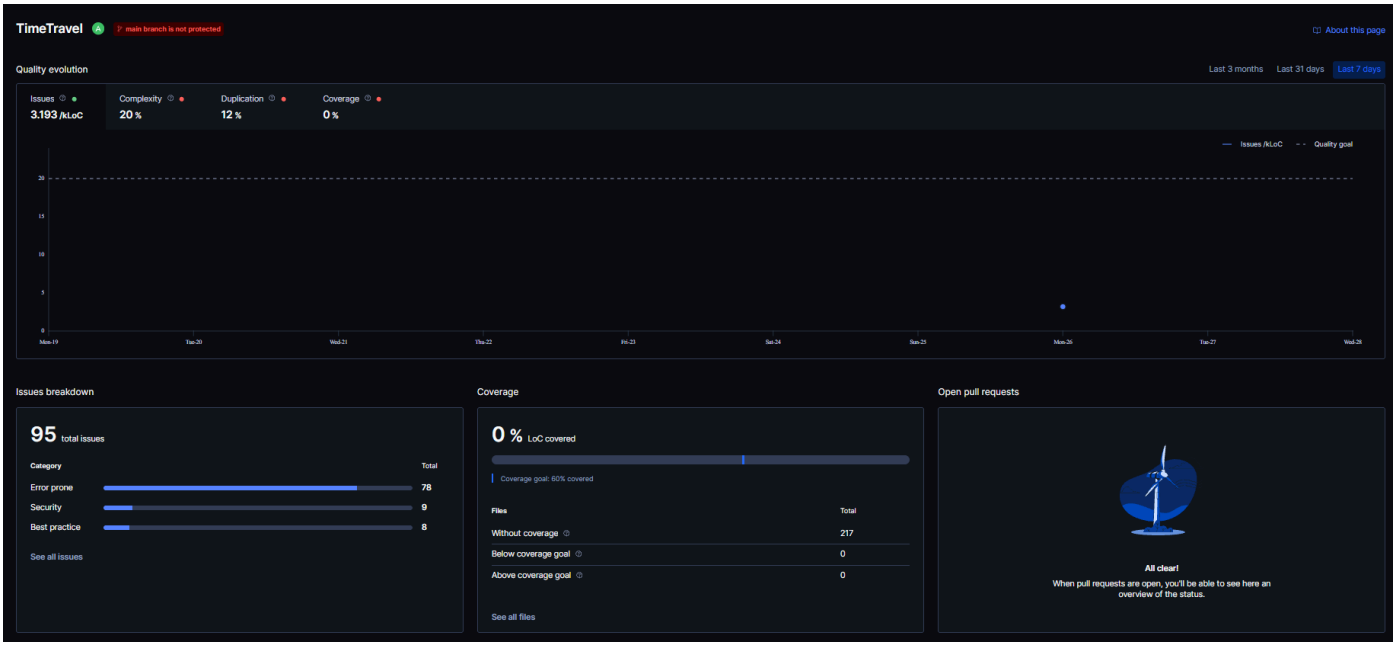
Locally, ESLint with the course's `.eslintrc.json` over `backend/src` and `backend/tests` reports 2 findings (explained in 2.4), and detekt 1.23.8 with the course's `detekt.yml` over `frontend/app/src` reports none.

2.1. Commit Hash Where Codacy Ran

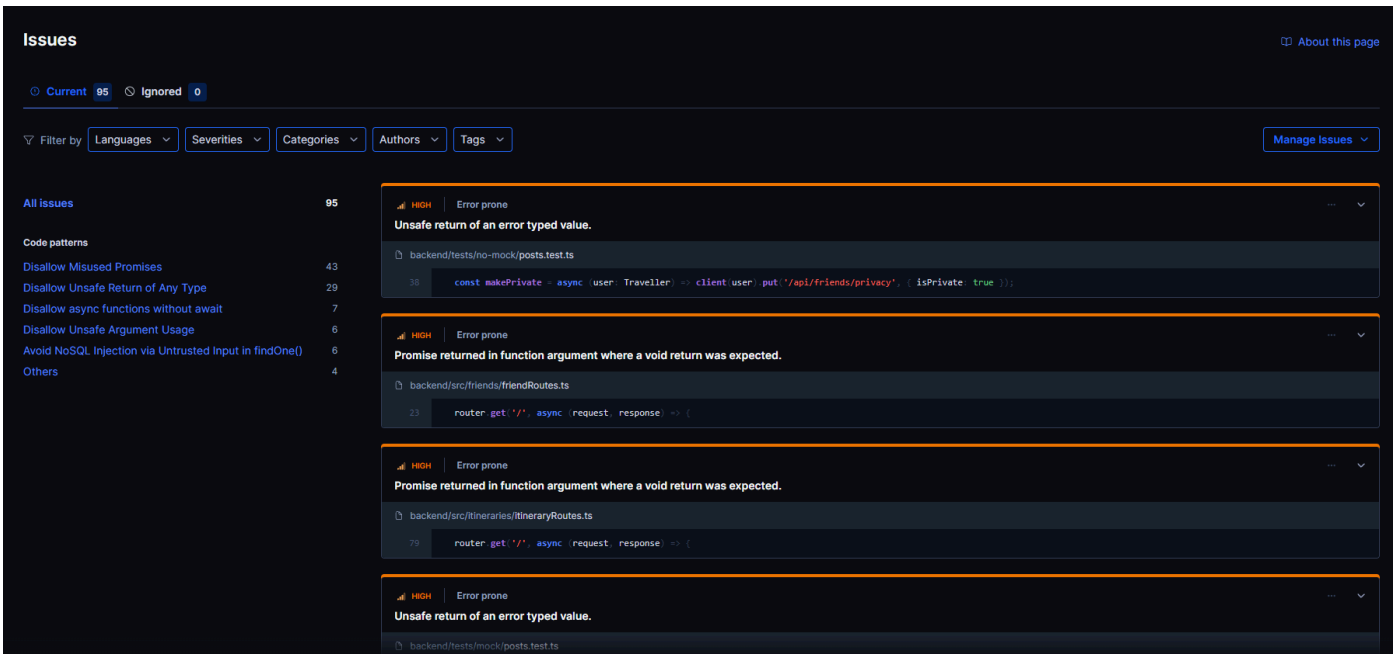
`ceec5d99d70ad8d0d26c8d107e0d631a3d966fd9`

2.2. Unfixed Issues per Codacy Category

95 unfixed issues: 78 Error Prone, 9 Security and 8 Best Practice.



2.3. Unfixed Issues per Codacy Code Pattern



The 4 issues under "Others":

Code pattern	Tool	Category	Issues
Enforce specifying a non-root USER in Dockerfile	Opengrep	Security	1
Avoid Server-Side Request Forgery (SSRF) by Validating User-Controlled URLs	Opengrep	Security	1
Detect Non-Literal Filename in fs Calls	ESLint	Security	1
Disallow Redundant Type Constituents in Unions and Intersections	ESLint	Best Practice	1

2.4. Justifications for Unfixed Issues

- **Error Prone and Best Practice: TypeScript rules that need type information** (ESLint), 86 issues:
  - `@typescript-eslint/no-misused-promises`, 43 issues: the `async` route handlers in `backend/src/*/*Routes.ts` (for example `groupRoutes.ts`).
  - `@typescript-eslint/no-unsafe-return`, 29 issues: test helpers in `backend/tests` that return a `supertest` request (for example `post: (path, body) => request(app).post(path)...`).
  - `@typescript-eslint/require-await`, 7 issues: `async` test helpers in `backend/tests` that return a `supertest` request or a promise.
  - `@typescript-eslint/no-unsafe-argument`, 6 issues: 4 in `backend/src/sync/tripSync.ts` (the Socket.io session's user id and acknowledgement callbacks), 1 in `backend/tests/helpers/testDatabase.ts#L14` and 1 in `backend/tests/nfr/liveUpdateLatency.test.ts#L65`.
  - `@typescript-eslint/no-redundant-type-constituents`, 1 issue: `let memoryServer: MongoMemoryServer | undefined` in `backend/tests/helpers/testDatabase.ts#L9`.
  - **Justification:** these rules read the types of values, but Codacy analyses the repository without installing its npm dependencies, so it treats the types from `express`, `supertest`, `socket.io` and `mongodb-memory-server` as `any`. Its own messages show this ("Unsafe return of an error typed value", and "'any' overrides all other types in this union type" for `MongoMemoryServer | undefined`). With the dependencies installed, the course's `.eslintrc.json`, which enables all five rules as errors, reports none of the 86 over `backend/src` and `backend/tests` on `ceec5d9`. With the real types, Express 5's handler type allows a returned promise (Express 5 passes a rejected one to the error handler), and a `supertest` request is promise-like, so returning it from an `async` helper is neither unsafe nor a missing `await`.
- **Security:** `rule-node-nosqli-injection` (Opengrep), 6 issues:
  - `backend/src/activities/activityService.ts#L176` and `#L180`, `backend/src/posts/postService.ts#L199` and `#L200`, `backend/src/friends/friendService.ts#L118` and `#L195`.
  - **Justification:** a NoSQL injection needs a request to put an object such as `{ "$ne": null }` into a query field, and none of these queries can receive one. Request ids are path parameters, which Express always parses as strings, and each is checked with `isValidObjectId` and converted to an `ObjectId` before the query (`activityId` at `activityService.ts#L142-L145`, `postId` and `commentId` at `postService.ts#L200` and `#L220`, the friend's id in `requireUser` at `friendService.ts#L187`). The other values are built by the server: a city key read from the stored activity, and the friendship key `pairKey(a, b)`, a string of two user ids.
- **Security:** `rule-node-ssrf` (Opengrep), 1 issue at `backend/tests/helpers/liveServer.ts#L31`
  - **Justification:** this is test code. The Socket.io client connects to the `http://localhost:<port>` address that `startLiveServer` returns for the test server it just started (`liveServer.ts#L16-L27`). No user input reaches the URL, and the file is not deployed.
- **Security:** `missing-user` (Opengrep), 1 issue at `backend/Dockerfile#L21`
  - **Justification:** the container runs as root because it reads the TLS key that the deployment mounts read-only from the VM (`docker-compose.yml`, `./backend/certs:/certs:ro`), and only the VM's own

user can read that key. Switching to the unprivileged `node` user needs the key's ownership changed on the VM, a new image and a redeploy, which we did not risk before the CP2 deadline. The exposure is limited: the container runs only the back-end and publishes only its port, MongoDB is not published outside the Docker network, and the certificates are read-only. Running as `node` is planned for the next deployment.

- **Security:** `security/detect-non-literal-fs-filename` (ESLint), 1 issue at `backend/src/index.ts#L50` (the course's ESLint configuration reports its two calls as 2 findings)
    - **Justification:** the server reads its TLS certificate and key with `readFileSync(env.tlsCertPath)` and `readFileSync(env.tlsKeyPath)`. The rule flags non-literal paths because a path built from user input could read any file. These paths come only from the server's `.env` (`TLS_CERT_PATH`, `TLS_KEY_PATH`), set by whoever deploys it, and are read once at start-up; no request can reach them. A literal path would tie the code to one server's layout (the deployment mounts the certificates at `/certs/...`).

---

3. Manual Code Review Results

Status at CP1: the peer review has not started. Artifacts are exchanged with the peer group by **November 2** (CP1 Progress, task 7), and the review report and this section are due **November 16** (task 10).

3.1. Review You Performed

- **Peer group reviewed:** *to be assigned*
- **Commit SHA reviewed:** *not yet exchanged*
- **Date the artifacts were exchanged:** *planned for November 2*
- **Review report submitted:** *not yet written*

The scores (manual code review, manual test review, automated code review) and the major fault will be recorded here after the review.

3.2. Review You Received

- **Reviewing group:** *to be assigned*
- **Commit SHA that was reviewed:** *not yet exchanged*
- **Review report received:** *not yet received*

The scores and the major fault reported against TimeTravel will be recorded here when the report is received.

3.3. Actions Taken in Response to the Review

None yet. Each finding from the review received, the action taken and the commit or location in Git will be listed here, together with a justification for any finding not acted on.